**Date handed out: 1st of April, 5 PM**
**Date submission due: 15th of April, 11 PM**

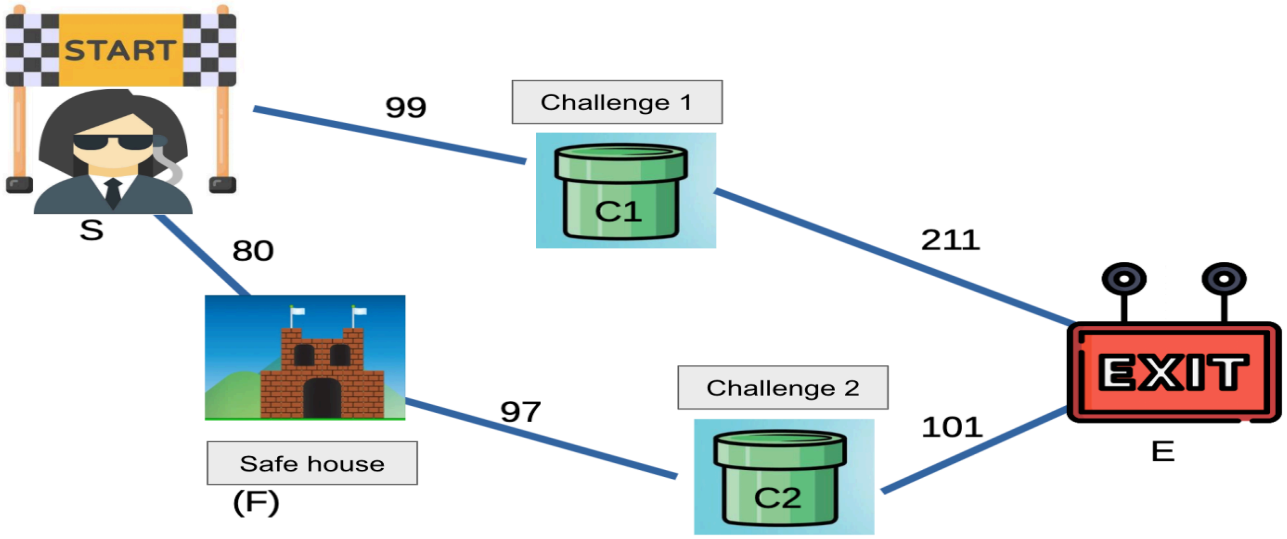
# Agent Lara's Rescue Mission

**The overall goal of this assignment is to enable you to practice how to use the search algorithms we learnt in class.**

***Important:*** **All the material included in this assignment could be included in future quizzes, midterms, and exams. Despite the workload distribution, I expect both team members to understand each other's tasks and the total code.**

---

Agent Lara is trapped on an island and must find her way to the exit (E) to survive. Although she has access to a map, the path is not straightforward. She must pass through two challenge rooms (C1 and C2), each testing her skills and endurance. Fortunately, a safe house (F) is located along the way, where she can rest and collect equipment to assist her escape. To increase her chances of success, she relies on your code, running it on a special device to simulate the path to freedom.




**Survival MAP**

## Task:

1. Use a proper data structure to represent and construct the previous map given to you in the form of a graph.

2. Apply UCS and BFS to find the path to exit the island. Your code must show the node expanded in every move. **You can apply these two algorithms manually and compare the result with your code**.

3. To help Lara make a decision which path to choose, answer the given questions. At the end of the program, your code must print the answers. An illustrative example of the expected output is shown below. In this example, the expanded node is masked with the symbol X; however, in your implementation, the actual letter of the expanded node should be printed.

**Questions (Task 1):**

- 1) Which path passes through the safe house (F)?

- 2) Which algorithm (BFS or UCS) reaches the goal with fewer expansions?

- 3) Does passing through the safe house always lead to an optimal solution? (Explain.)

- 4) Compare the solution paths returned by BFS and UCS.

***Illustrative output example :***

Welcome to Agent Lara's Rescue Mission —> choose the Algorithm to Apply:

1) UCS
2) BFS

Choice: 1

Expanding X

Expanding X

Expanding X.. Moving to challenge X

Solution path XXXX

*** Now the second algorithm will be applied ***

Expanding X

Expanding X

Expanding X.. Moving to challenge X

Solution path XXXX

***Answers ***

Q1 ……

Notice how, in the illustrative output example, there is a message indicating that Lara is moving to a specific challenge when any of the search algorithms is used. This happens because C1 and C2 represent challenges that Lara must solve. If Lara successfully overcomes these challenges, she can proceed to expand the next node, bringing her closer to the exit. However, if she fails, the program should display a message stating that she did not survive the challenge and must exit the game. That said, failure should not occur, as these challenges are well-defined problems. With the right search algorithms, Lara should always be able to find a solution and continue making progress toward the goal.

**Challenge Room 1: Guards and Prisoners Problem**

This is a classic AI search problem in which three guards and three prisoners must cross a river using a boat that can carry at most two people at a time. At all times, the number of prisoners must not exceed the number of guards on either side of the river; otherwise, the prisoners would overpower the guards. Each state is represented as a tuple **(G, P, B)**, where **G** and **P** indicate the number of guards and prisoners on the left bank, and **B** represents the position of the boat (**0 for left bank, 1 for right bank**). The objective is to transform the system from the initial state **(3,3,1)** to the goal state **(0,0,1)** by applying valid actions (moving one or two individuals at a time) while always maintaining safe configurations.

**Requests:**

1) Formulate the problem correctly using proper problem formulation items. The formulation should be in the ReadMe file and should match your code. **Hints**:
    - Boat capacity: 2
    - Prisoners must never outnumber guards on either side
    - State representation: (G, P, B)

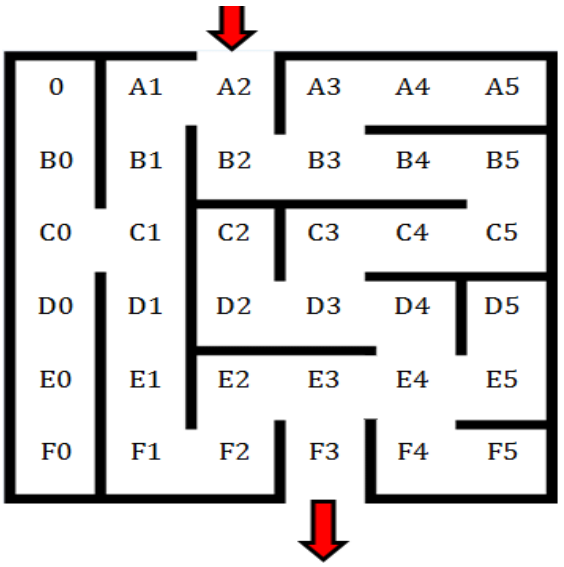
2) Apply the A* algorithm so Lara can solve the challenge correctly and move to the next node to expand in the previous map.
3) Remember, A* prioritizes states with the lowest f(n) = g(n) + h(n) value, so you need to find a proper Heuristic Function (Must be admissible).
4) Your code must return the solution path and show the expansion order.
5) Similarly, your code must provide answers to the following questions. Note that the answers must be specific to the given problem and not generic responses.

**Questions (Challenge 1):**

- What makes a state invalid in this problem?
- Why must the heuristic be admissible?
- What happens if your heuristic overestimates?

**Challenge 2:Labyrinth Search**

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 1 | A0 | A1 | A2 | A3 | A4 | A5 |
| 2 | B0 | B1 | B2 | B3 | B4 | B5 |
| 2 | C0 | C1 | C2 | C3 | C4 | C5 |
| 3 | D0 | D1 | D2 | D3 | D4 | D5 |
| 4 | E0 | E1 | E2 | E3 | E4 | E5 |
| 5 | F0 | F1 | F2 | F3 | F4 | F5 |




Lara always starts in room A2 (the maze entrance) and must locate a mad scientist, who can be in any room except the entrance (A2). Once Lara finds the mad scientist, they must try to reach the exit at F3. The maze structure is provided in a .txt file containing data that describes the layout. For example, a file may include the line: A2 1 2 A1, indicating that room A2 is at position (1,2) and is connected to room A1. The last lines of the file specify the mad scientist's location and the exit at F3. To test your code, you can use the provided map.

**Example (for illustrative purposes):**

A2 0 2 A1 B2
A1 0 1 B1 A2
B1 1 1 A1 C1
…..
……
mad scientest F5
Exit F3

**Request:**

1) Out of the txt file, construct a graph; the file must be submitted with the assignment for testing purposes.
2) Apply the Greedy best-first algorithm **once to find the mad scientist and once to find the Exit,** using the explained **Manhattan distance between two points as the heuristic**. Make sure to apply Graph search where nodes are never expanded twice.
3) Similarly, your code must provide answers to the following questions. Note that the answers here can be generic responses and not specified to the problem at hand.

**Questions (Challenge 2):**

- Is Greedy Best-First Search guaranteed to find the optimal?
- How does the heuristic affect node expansion?
- What happens if nodes are expanded more than once?

## Programming Languages:

You can use Python or C.

## References:

***Maze:***
https://www.researchgate.net/publication/315969093_Maze_Exploration_Algorithm_for_Small_Mobile_Platforms

| Grading | Points |
|---|---|
| Use a proper Data Structure to represent the map in task 1, and properly integrate challenges 1 and 2 in the code of task 1 | 10 |
| BFS and UFC were correctly applied in task 1 | 10 (5 per each) |
| Show the expansion order as the search is applied for BFS and UFC | 15 (7.5 per each) |
| Challenge 1 correctly applied | 10 (problem formulation, proper heuristic choice, A* correctly working) |
| shows the expansion order of A* in challenge 1, returning the solution path | 15 |
| Challenge 2 was correctly applied | 15 (reading map from file, reading goal from the file, properly constructing the search graph, and applying greedy first) |
| Show the expansion order of Greedy in Challenge 2, returning the solution path | 15 |
| Proper coding | Up to 10% of the total grade may be deducted for not following proper coding practices. |
| Answering the questions | 10 |